MASTER OF SCIENCE IN ENGINEERING

Hes·so Master

HES-SO Master

Master in Engineering - Computer Science

Environnement Linux embarqué et programmation noyau Linux

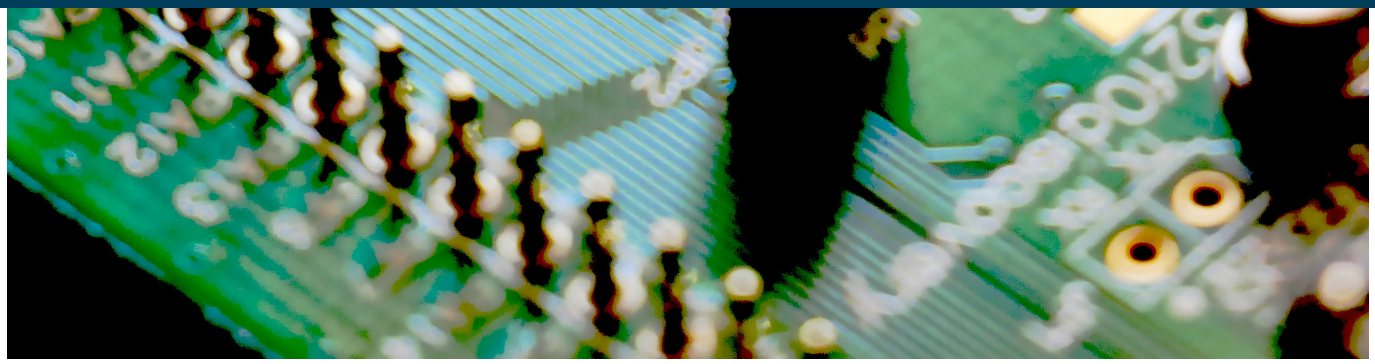
CSEL - Construction de Systèmes Embarqués sur Linux

Auteurs

Carolina Inácio Pereira, Gaby Roch, Simon Mirkovitch

Dépot Git

https://github.com/g-roch/mse-csel-workspace



Rapport de laboratoire 1,
2 Avril 2026

Hes·so

Table des matières

1 Introduction . . . 2
2 Environnement Linux embarqué . . . 3
2.1 Résumé du laboratoire . . . 3
2.2 Questions . . . 3
2.3 Ce qui a été appris / exercé . . . 5
2.4 Remarques / À retenir . . . 5
2.4.1 Distinction entre les configurations Buildroot et Noyau Linux . . . 5
2.5 Feedback personnel . . . 6
2.5.1 Carolina . . . 6
2.5.2 Gaby . . . 6
2.5.3 Simon . . . 6
3 Programmation noyau . . . 7
3.1 Résumé du laboratoire . . . 7
3.2 Questions . . . 7
3.3 Ce qui a été appris / exercé . . . 7
3.3.1 Partie « Modules noyaux . . . 7
3.3.2 Partie « Pilotes de périphériques » . . . 8
3.4 Remarques / À retenir . . . 9
3.4.1 Application utilisant les pilotes orientés caractère . . . 9
3.5 Feedback personnel . . . 9
3.5.1 Carolina . . . 9
3.5.2 Gaby . . . 9
3.5.3 Simon . . . 9
4 Annexes . . . 10
A. Application en Rust . . . 10

1 Introduction

Ce rapport présente les travaux réalisés lors des laboratoires consacrés à l'environnement Linux embarqué et à la programmation noyau Linux.

La première partie vise à mettre en place un environnement de développement complet pour un système embarqué, incluant la configuration de Buildroot, la génération de l'image système, du noyau et du root filesystem, ainsi que la compréhension des différentes stratégies de boot (carte SD, eMMC, mode CIFS/SMB).

La seconde partie porte sur le développement de modules noyau et de pilotes de périphériques, en abordant notamment la création de modules *out of tree*, la gestion de la mémoire et des threads dans le noyau, la lecture directe de registres processeur et la mise en place de pilotes orientés mémoire et caractère, complétée par une application utilisateur écrite en Rust permettant de tester ces interfaces.

L'objectif de ce rapport est de résumer les étapes réalisées, de synthétiser les concepts appris et de partager les remarques et retours d'expérience associés à ces laboratoires.

2 Environnement Linux embarqué

2.1 Résumé du laboratoire

Ce laboratoire consistait principalement à mettre en place l'environnement de développement. La marche à suivre donnée était claire et facile à suivre. La compilation des images peut prendre un certain temps, mais une fois la première compilation faite, les suivantes sont beaucoup plus rapides.

2.2 Questions

Question 1
Comment faut-il procéder pour générer l'U-Boot ?

Le Buildroot fourni intégrait déjà U-Boot. Pour l'ajouter nous-mêmes il faut d'abord configurer le Buildroot :

```
cd /buildroot
make menuconfig
```

Ensuite choisir `Bootloaders > U-Boot` et enregistrer les modifications.

Enfin, lancer la commande `make` pour télécharger, compiler et intégrer U-Boot dans l'image.

Question 2
Comment peut-on ajouter et générer un package supplémentaire dans le Buildroot ?

Pour ajouter un package supplémentaire dans le Buildroot, il faut le configurer avec :

```bash
cd /buildroot
make menuconfig
```

Dans `Target packages`, activer le package souhaité. Ensuite, lancer la commande `make` pour télécharger, compiler et intégrer le paquet au système.

Ne pas oublier de mettre à jour le *rootfs (root filesystem)* :

```bash
/usr/local/bin/extract-rootfs.sh
```

Question 3
Comment doit-on procéder pour modifier la configuration du noyau Linux ?

Pour modifier la configuration du noyau Linux, il faut lancer la commande suivante :

```bash
cd /buildroot
make linux-menuconfig
```

Il est important de relancer `make` pour recompiler avec les modifications apportées.

Question 4
Comment faut-il faire pour générer son propre rootfs ?

Le rootfs est généré automatiquement par Buildroot lors du `make`. Pour le personnaliser :

1. `make menuconfig` ⟶ `Target packages` (choisir BusyBox, Python, etc.)
2. `Filesystem images` ⟶ choisir format (ext4, squashfs, tar)
3. `System configuration` ⟶ init system, users, hostname
4. Optionnel : ajouter fichiers dans `board/<nom>/rootfs-overlay/`
5. `make` ⟶ génère `output/images/rootfs.ext4`
6. `extract-rootfs.sh` ⟶ extrait dans workspace pour modification

Question 5
Comment faudrait-il procéder pour utiliser la carte eMMC en lieu et place de la carte SD ?

Les étapes sont les suivantes :
1. Graver `sdcard.img` sur la carte SD et démarrer
2. Copier `sdcard.img` sur l'eMMC :

```bash
dd if=sdcard.img of=/dev/mmcblk1 bs=1M
```

3. Dans le script de démarrage pour U-Boot : changer `mmc 0` à `mmc 1`
4. Redémarrer pour laisser U-Boot lire les nouveaux scripts et démarrer depuis l'eMMC

Question 6
Dans le support de cours, on trouve différentes configurations de l'environnement de développement. Quelle serait la configuration optimale pour le développement uniquement d'applications en espace utilisateur ?

La meilleure configuration est le mode boot CIFS/SMB :

- Le noyau est chargé depuis la carte SD : l'opération est plus rapide et stable comparée au chargement via le réseau avec TFTP.
- Parce que le rootfs est monté en CIFS depuis l'hôte, les modifications en espace utilisateur sont immédiatement disponibles sur la cible.

2.3 Ce qui a été appris / exercé

Le tableau ci-dessous présente les compétences exercées durant les laboratoires selon trois niveaux d'acquisition : *non acquis*, *acquis, mais à exercer* et *parfaitement acquis*.

Point / Sujet	Carolina	Gaby	Simon
Mise en œuvre d'un système embarqué sous Linux	Acquis, mais à exercer	Parfaitement acquis	Parfaitement acquis
Mise en œuvre de l'environnement de développement de systèmes embarqués sous Linux avec Docker	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Debugging d'applications sous Linux embarqué	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Mise en production d'un système embarqué sous Linux	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer

2.4 Remarques / À retenir

2.4.1 Distinction entre les configurations Buildroot et Noyau Linux

Pendant les laboratoires, nous avons utilisé la commande pour configurer le Buildroot :

```bash
make menuconfig
```

La question 3 nous a permis de découvrir que la configuration du noyau Linux se fait séparément avec :

```bash
make linux-menuconfig
```

Cette confusion vient du fait que les rôles de Buildroot et du noyau Linux n'ont pas été bien compris :

- Buildroot : outil permettant d'automatiser la création de systèmes Linux embarqués. À partir d'une configuration, il génère :
  - une toolchain croisée
  - le noyau Linux
  - le bootloader
  - le système de fichiers racine (root filesystem)
- Noyau Linux (kernel) : le cœur du système d'exploitation. Il gère le matériel et offre des services aux applications. Sans lui, aucune interaction avec le matériel n'est possible.

Différences entre les outils de configuration :

- `menuconfig` (Buildroot) permet de configurer :
  - les bootloaders (U-Boot)
  - les packages (BusyBox, SSH, Python...)
  - le type du système de fichiers racine (ext4, squashfs)
  - la toolchain (GCC version)
- `linux-menuconfig` (kernel) permet de configurer le noyau lui-même, notamment :
  - les drivers matériels (USB, Ethernet, GPIO)
  - les systèmes de fichiers supportés (ext4, nfs, cifs)
  - le réseau (TCP/IP stack)
  - la gestion des processus et de la mémoire

Donc, à retenir simplement :
- `menuconfig` configure le système global (espace utilisateur + build)
- `linux-menuconfig` configure l'espace noyau

2.5 Feedback personnel

2.5.1 Carolina

J'ai encore certaines difficultés avec les systèmes embarqués, ce qui fait que le rythme du cours peut parfois me sembler rapide et les laboratoires longs à assimiler.

Ce laboratoire était relativement simple à compléter grâce aux instructions claires et faciles à suivre. Cependant, le fait que les solutions soient déjà disponibles m'a permis d'avancer sans toujours bien comprendre les mécanismes sous-jacents.

C'est finalement lors de la rédaction de ce rapport que j'ai pu clarifier des notions importantes comme le rôle de Buildroot, du noyau Linux et du root filesystem. Une amélioration possible serait de rendre les solutions accessibles sans qu'elles soient directement intégrées, pour encourager les élèves à essayer de faire eux-mêmes les exercices.

En résumé, j'ai apprécié la clarté des instructions pour ce premier laboratoire, la possibilité de ne pas rester bloqué grâce aux solutions, ainsi que la rédaction du rapport, qui oblige à réellement comprendre le travail effectué.

2.5.2 Gaby

Pas de commentaire particulier sur ce laboratoire.

2.5.3 Simon

Ayant déjà vu une bonne partie de la matière durant mon Bachelor à l'Heig-VD, ce fut surtout de la révision pour ma part. Par contre, les explications ont été bien plus claires que celles que j'avais eu précédemment et vont bien plus dans les détails. Ce qui fait que j'ai appris des choses que je n'avais jamais vu. Par exemple: les différentes configurations de l'environnement (on avait utilisé seulement TFTP), les instruments de Debug ou encore les `dev-container` avec Docker qui étaient une totale nouveauté. Les questions étaient aussi très pertinentes et m'ont permis d'en apprendre plus.

3 Programmation noyau

3.1 Résumé du laboratoire

Ce laboratoire se divise en deux aspects fondamentaux du développement noyau Linux :

- **Modules noyau** :
  - Création de modules *out of tree*
  - Gestion avec `modprobe`/`insmod`
  - Accès aux registres microprocesseur (Chip-ID, température, MAC)
  - Implémentation de threads noyau avec `kthread_run`
  - Synchronisation via *waitqueues*
  - Gestion des interruptions GPIO (boutons carte d'extension)
- **Pilotes de périphérique** :
  - Développement de pilotes orientés mémoire et caractère
  - Création de plusieurs instances via numéros *minor*
  - Intégration *sysfs* (`/sys/class/...`)
  - Interface *platform bus*
  - Application utilisateur testant les pilotes via `/dev/mymodule.X`

3.2 Questions

Ce laboratoire ne contenait aucune question.

3.3 Ce qui a été appris / exercé

3.3.1 Partie « Modules noyaux

Point / Sujet	Carolina	Gaby	Simon
Génération d'un module noyau « out of tree »	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Gestion d'un module noyau avec modprobe	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Création dynamique d'éléments dans un module noyau	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Lecture des registres du microprocesseur dans un module noyau	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer

Point / Sujet	Carolina	Gaby	Simon
Création et gestion de threads dans un module noyau	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Mise en sommeil et réveil de threads dans un module noyau	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Gestion des interruptions dans un module noyau	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer

3.3.2 Partie « Pilotes de périphériques »

Les tableaux ci-dessous présentent les compétences exercées durant les laboratoires selon trois niveaux d'acquisition : *non acquis*, *acquis, mais à exercer* et *parfaitement acquis*.

Point / Sujet	Carolina	Gaby	Simon
Réalisation d'un pilote orienté mémoire	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Réalisation d'un pilote orienté caractère simple	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Intégrer un pilote orienté caractère dans un module noyau	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Développement d'une application en espace utilisateur utilisant un ou plusieurs pilotes orientés caractère	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Réalisation d'un pilote orienté caractère avec *sysfs* (class)	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer
Réalisation d'un pilote orienté caractère avec le bus *plateforme*	Acquis, mais à exercer	Acquis, mais à exercer	Acquis, mais à exercer

3.4 Remarques / À retenir

3.4.1 Application utilisant les pilotes orientés caractère

Dans le cadre de l'exercice 4 de la partie « Pilotes de périphériques », une application a été développée en Rust (code source : Annexe A.).

Cet exercice a permis à l'étudiante de se familiariser pour la première fois avec ce langage.

3.5 Feedback personnel

3.5.1 Carolina

Ce laboratoire m'a encouragée à travailler de manière plus autonome, étant donné que nous avons été moins guidées que lors du premier. J'ai par exemple passé beaucoup de temps sur un exercice sans savoir qu'une solution existait déjà, ce qui m'a fait prendre conscience de l'importance de bien lire les consignes et de vérifier rapidement si une aide est disponible.

Il m'a permis de mieux comprendre la création et la gestion de modules noyau (`insmod`, `modprobe`, `dmesg`, `modinfo`, gestion mémoire), ainsi que l'accès direct aux registres du processeur, la gestion de threads noyau et de *waitqueues*, ce qui m'a clarifié comment le noyau contrôle le matériel.

Ce travail m'a également donné l'occasion de découvrir le langage de programmation Rust en développant un petit programme destiné à tester les pilotes noyau pour l'exercice 4 de la partie « Pilotes de périphériques ».

3.5.2 Gaby

Pas de commentaire particulier sur ce laboratoire.

3.5.3 Simon

Cette partie était aussi de la révision, car la matière avait été entièrement vue durant mon Bachelor. Cela dit, j'ai quand même dû bien réfléchir, car je n'avais pas pratiqué ce sujet depuis que j'ai eu mon diplôme. Je ne suis pas certain de savoir le fonctionnement exact de chaque composant, mais, avec la « boîte à outils » fournie dans le cours, il m'a été possible de me remettre dans le bain.

4 Annexes

A. Application en Rust

Application développée pour l'exercice 4 du laboratoire « Pilotes de périphériques ».

```rust
// Import necessary modules
// - File and OpenOptions for file handling
use std::fs::{File, OpenOptions};
// - Read and Write traits for reading and writing to files
use std::io::{Read, Write};

// HELPER FUNCTIONS //
fn write_to_file(file_path: &str, content: &str) -> std::io::Result<()> {
    // Create or open the file for writing
    let mut file = OpenOptions::new()
        .write(true)
        .create(true)
        .open(file_path)?;

    // Write the content to the file
    file.write_all(content.as_bytes())?;
    Ok(())
}

fn read_from_file(file_path: &str) -> std::io::Result<String> {
    // Open the file for reading
    let mut file = File::open(file_path)?;
    let mut content = String::new();

    // Read the content of the file into a string
    file.read_to_string(&mut content)?;
    Ok(content)
}
// ---------------- //

// Define the main function
fn main() {
    // Define the device paths (these should match the ones created by your kernel
    module)
    let devices = vec!["/dev/mymodule", "/dev/mymodule.1", "/dev/mymodule.2"];
    // Define the content to write to each device
    let contents = vec![
        "Hello device 0 from Rust!",
        "Hello device 1 from Rust!",
        "Hello device 2 from Rust!",
```

```
    ];

    // Loop through each device and perform write and read operations
    for (i, device) in devices.iter().enumerate() {
        // Write to the device
        match write_to_file(device, contents[i]) {
            Ok(_) => println!("Content successfully written to {}", device),
            Err(e) => eprintln!("Failed to write to {}: {}", device, e),
        }

        // Read from the device
        match read_from_file(device) {
            Ok(content) => println!("Content read from {}: {}", device, content),
            Err(e) => eprintln!("Failed to read from {}: {}", device, e),
        }
    }
}
```